THE UNIVERSITY OF DA NANG

UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF ADVANCED SCIENCE AND TECHNOLOGY

COURSE PROJECTS

COURSE'S TITLE: DIGITAL LOGIC CIRCUIT

PROJECT'S TITLE: COMBINATION DIGITAL LOCK

Supervisor: Nguyễn Quang Như Quỳnh

Students:

Vũ Nguyên Hồng Sơn	ID: 123230086
Trương Duy Bảo	ID: 123230055
Trần Lê Quốc Hưng	ID: 123230066
Nguyễn Đăng Khoa	ID: 123230072
Nguyễn Thị Mỹ Duyên	ID: 123230059

Class: 23ECE

Da Nang, 5/2025

TABLE OF CONTENTS

Page

I. Introduction……2

II. Division of work……2

III. Decimal to BCD Encoding – BCD Code……3

IV. Memory input and JK Flip-Flop (write control unit)……6

V. Password Memory Blocks……9

VI. Binary Decoder Block……10

VII. Comparator……12

VIII. Simulation and applications……12

IX. Conclusion……13

X. Time schedule…… 13

REFERENCES……14

I. Introduction

- The digital combination lock is a simple yet effective security solution widely used in practice. This system uses basic digital circuit blocks such as an encoder, memory, JK flip-flop, comparator, and 7-segment display.

II. Division of work

1. **Nguyễn Thị Mỹ Duyên:** Working on the Decimal to BCD encoder Block.
2. **Nguyễn Đăng Khoa:** Working on the Memory input and JK Flip-Flop Block.
3. **Trương Duy Bảo:** Working on the Password Memory Block.
4. **Trần Lê Quốc Hưng:** Working on the BCD to 7-segment Display Decoder Block.
5. **Vũ Nguyên Hồng Sơn:** Working on the Comparator Block and simulation.

III. Decimal to BCD Encoding – BCD Code (Binary-Coded Decimal)

1. Decimal to BCD Encoding – BCD Code (Binary-Coded Decimal)

- A decimal to BCD encoder is a combinational circuit that converts each decimal digit into its corresponding BCD code. It is a binary encoding system in which each decimal digit is represented by a four-bit binary number.

Decimal Number	BCD or 8421 Code
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Example:

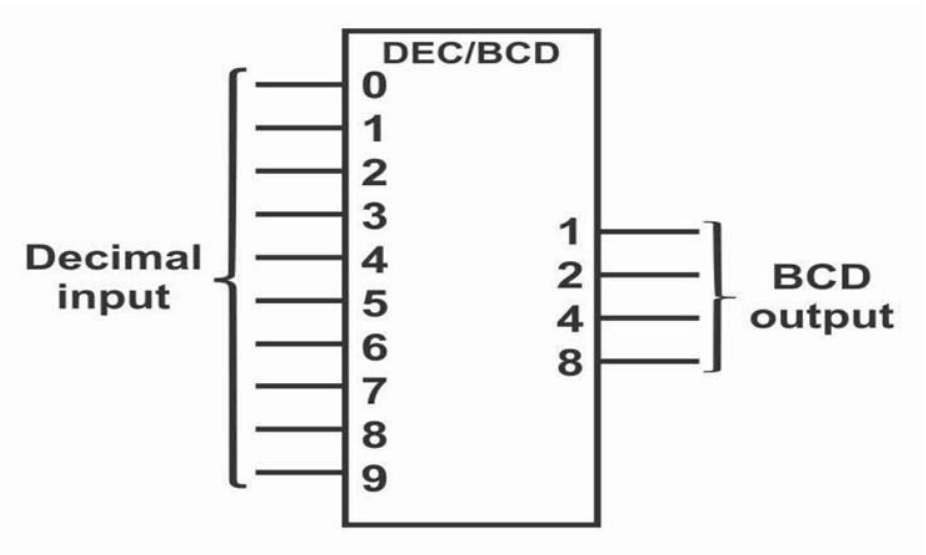
- The decimal number 0 is represented in BCD as **0000**.
- The decimal number 1 is represented in BCD as **0001**.

NOTE:
BCD only encodes decimal digits from 0 to 9. It is not used to represent decimal numbers greater than 9 directly with a single group of four bits. Instead, each digit of a larger decimal number is encoded individually into BCD.

Example:
The decimal number 12 is represented in BCD by encoding the digit **1** as **0001** and the digit **2** as **0010**, so 12 in BCD is **0001 0010**.

2. Decimal to BCD Encoder
This encoder has ten inputs, corresponding to the decimal digits from 0 to 9. It has four outputs that represent the BCD code.

Purpose of the Decimal to BCD Encoder:
To convert the active decimal input into its BCD (Binary-Coded Decimal) representation at the outputs



From the TRUTH TABLE:

Input										Output			
D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0	1	0	0	1

We have 4 outputs: (using minterm)

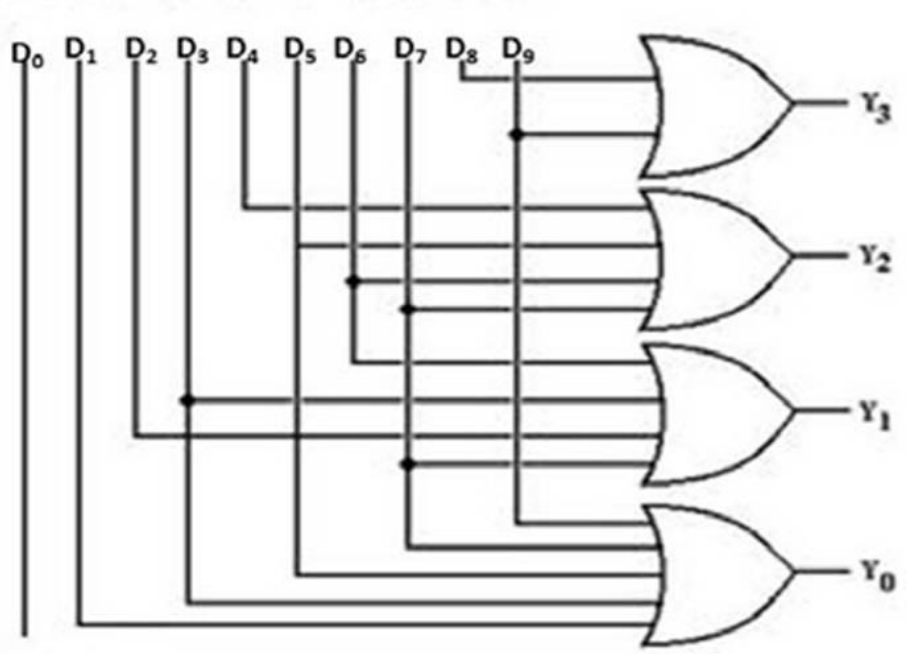
$$Y_0 = D_1 + D_3 + D_5 + D_7 + D_9$$

$$Y_1 = D_2 + D_3 + D_6 + D_7$$

$$Y_2 = D_4 + D_5 + D_6 + D_7$$

$$Y_3 = D_8 + D_9$$

We have the expressions for each output, so we can implement the decimal to BCD encoder using OR logic gates.

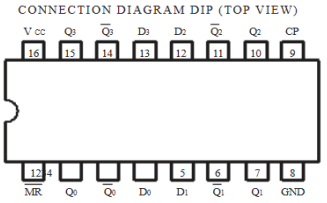


IV. Memory input and JK Flip-Flop (write control unit)

1. Memory input:

- Memory input is a temporary storage used to hold the values being entered by the user, which are then compared with the stored password.
- Idea: The memory input will consist of D flip-flops (each storing 1 bit of information), and the user will enter 2 numbers to unlock the system. Therefore, we need 8 D flip-flops to store the 2 numbers.
- However, for the convenience of simulation and to ensure signal synchronization, our group will use two 74LS175 ICs to store the two input numbers.

2. IC 74LS175

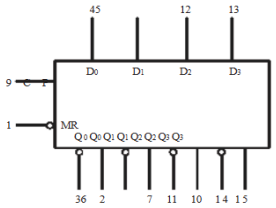


NOTE:
The Flatpak version has the same pinouts (Connection Diagram) as the Dual In-Line Package.

PIN NAMES		LOADING (Note a) HIGH	LOW
$D_0 - D_3$	Data Inputs	0.5 U.L.	0.25 U.L.
CP	Clock (Active HIGH Going Edge) Input	0.5 U.L.	0.25 U.L.
$\overline{MR}$	Master Reset (Active LOW) Input	0.5 U.L.	0.25 U.L.
$Q_0 - Q_3$	True Outputs	10 U.L.	5 U.L.
$\overline{Q}_0 - \overline{Q}_3$	Complemented Outputs	10 U.L.	5 U.L.

NOTES:
a) 1 TTL Unit Load (U.L.) = 40 μA HIGH/1.6 mA LOW.

LOGIC SYMBOL



VCC = PIN 16
GND = PIN 8

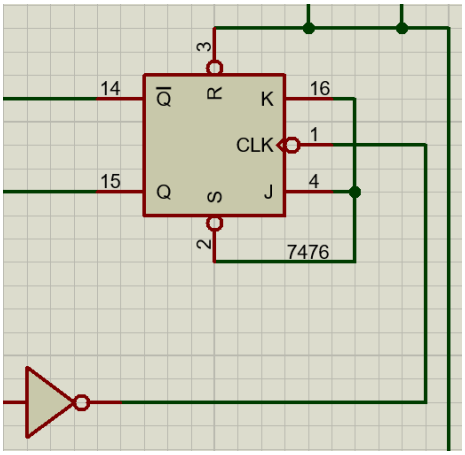
- The 74LS175 is a quad D-type flip-flop commonly used to store binary data synchronously with the clock signal. It is a TTL (Transistor-Transistor Logic) IC that contains four independent D flip-flops, all triggered on the rising edge of the clock.
- The IC has the following important pins:
    - D0–D3: Data input pins (4-bit BCD input)
    - Q0–Q3: Data output pins (output of stored data)
    - CLK: When a leading edge occurs, the values on D0–D3 are latched into Q0–Q3.
- Operation principle :

- You use 10 switches to input a number (BCD 0–9), which is fed into D0–D3.

- When you press the button:
  + A clock pulse is generated.
  + On the rising edge (as specified in the datasheet, this IC is positive edge triggered),
    – That is, when the CLK signal transitions from 0 to 1, the data on D0–D3 is latched into Q0–Q3.
  → The IC captures the values currently present on D0–D3 and stores them in its flip-flops.

- After being latched, the outputs Q0–Q3 hold the stored value stably until the next CLK pulse occurs.

3. JK Flip-Flop (write control unit)



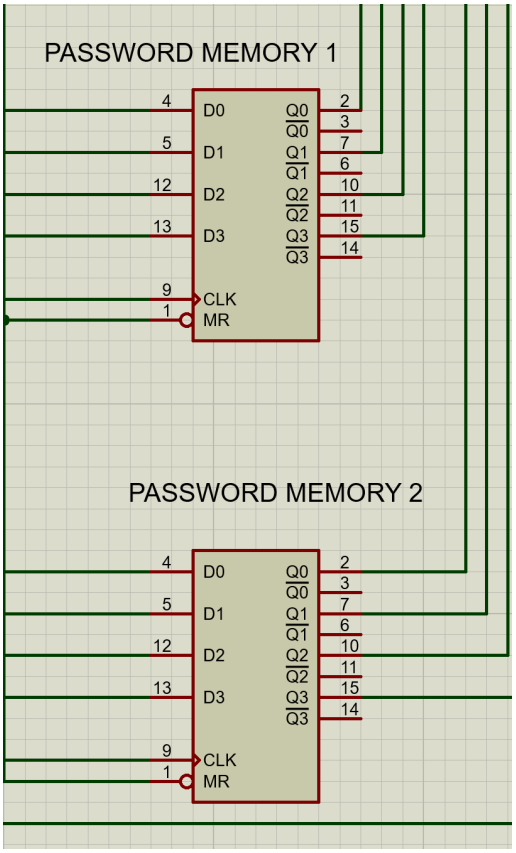
- The write control unit using a JK flip-flop is a part of the circuit responsible for determining when data is written into the 74LS175 memory ICs. It operates based on the characteristic toggling behavior of the JK flip-flop.
- The JK flip-flop here functions as a switch to control the write enable signal for memory:
    - When Q = 1: data is allowed to be written into memory.
    - When Q = 0: writing is disabled (the memory retains its previous value)

→ The flip-flop controls the “write or no write” state each time the button is pressed.

- Operation principle:
    - J = K = 1 → operates in toggle mode.
    - Each time the button is pressed, the JK flip-flop receives a pulse → toggles its Q state (from 0 → 1 → 0, etc.).
    - The Q output of the JK flip-flop is connected to the CLK input of the 74LS175 (either directly or through an AND gate with the button pulse).
    - When Q = 1 and a pulse from the button is present: data on D is latched into Q of the 74LS175.
    - When Q = 0: the clock path is blocked, and the 74LS175 does not perform a write operation.

V. Password Memory Blocks

- PMB is used to store and verify the password that the user has entered into the system.



Function:

1. **Storage of Passwords:**
   - These blocks store the password in binary form.
   - One for the current saved the first number of the password.
   - Another to save the second number of the password.
2. **Comparison with User Input:**
   - When the user enters a password, it is held in the Memory Block and passed to the 8-bit Comparator Block.
   - The comparator checks the user input against the stored password in the PMB.
   - If the two values are matched, the comparator sends a signal to trigger the Unlock Lock Block in the circuit.

How does it work?

1. **Password Entry:**
   - The user inputs a password using a keypad or decimal-to-binary input module.
   - The entered digits are stored in the Memory Blocks and converted into binary.
2. **Comparison:**
   - The 8-bit Comparator block compares the input from the Memory Blocks with the stored password from the Password Memory Blocks.
3. **Decision:**
   - **If those signals are matched:** the comparator enables the Unlock Lock Block, unlocking the system.
   - **If those are not matched:** the 8-bit Comparator Block behaves nothing.
4. **Password Update:**
   - If the "New Pass" button is triggered, the current input is stored in the Password Memory Block, updating the system's access code.

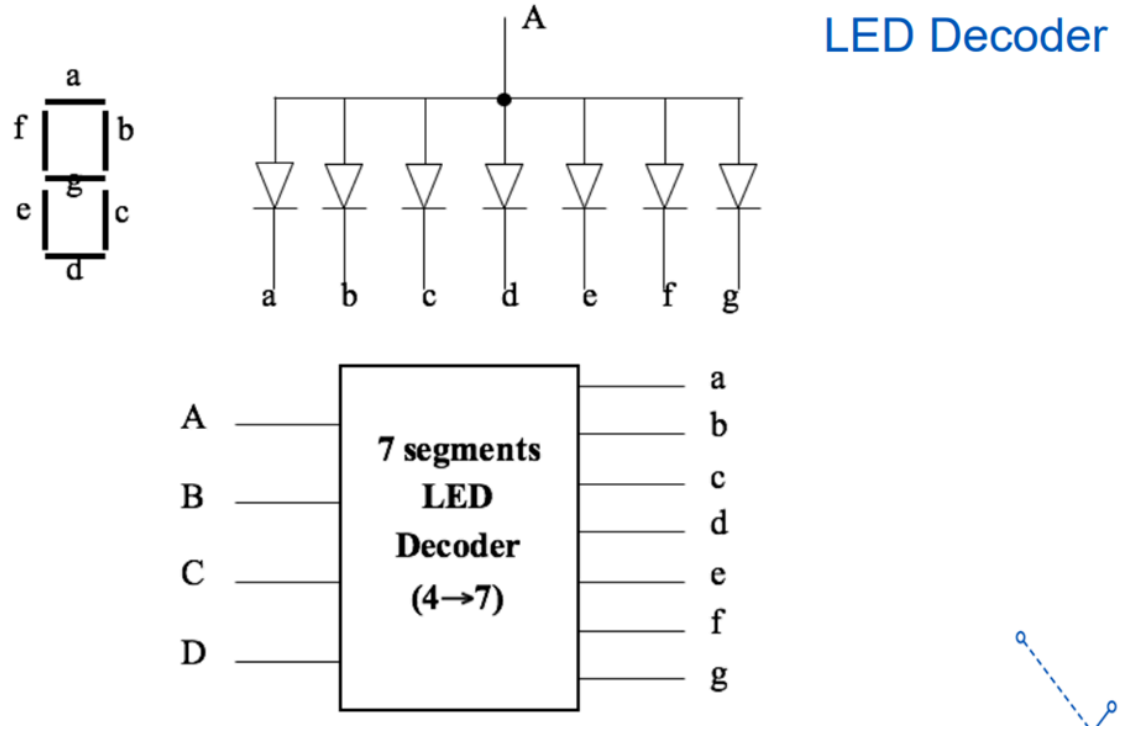
VI. Binary Decoder Block

- A 7-segment display is used to visually represent numbers (0-9) and some characters.

- A decoder converts binary input (usually 4 bits) into signals that light up the correct segments.

- The goal: Display decimal digits using binary input (0-9 in BCD).

1. How does the decoder work?

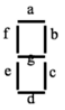


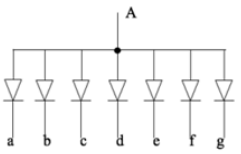
- Input: 4-bit Binary Coded Decimal (BCD) - values 0000 to 1001 (0 to 9).
- Output: 7 control signals for segments ‘a’ to ‘g’
- Each binary input combination activates a specific pattern of segments to show a digit.

2. Type of decoder

Common Anode LED

D	C	B	A	a	b	c	d	e	f	g	Number
0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	1	0	0	1	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0	2
0	0	1	1	0	0	0	0	1	1	0	3
0	1	0	0	1	0	0	1	1	0	0	4
0	1	0	1	0	1	0	0	1	0	0	5
0	1	1	0	0	1	0	0	0	0	0	6
0	1	1	1	0	0	0	1	1	1	1	7
1	0	0	0	0	0	0	0	0	0	0	8
1	0	0	1	0	0	0	0	1	0	0	9
1	0	1	0	X	X	X	X	X	X	X	X
1	0	1	1	X	X	X	X	X	X	X	X
1	1	0	0	X	X	X	X	X	X	X	X
1	1	0	1	X	X	X	X	X	X	X	X
1	1	1	0	X	X	X	X	X	X	X	X
1	1	1	1	X	X	X	X	X	X	X	X





- Example of output “a” using Maxterm to calculate

K-map for a

CD \ AB	00	01	11	10
00	1	0	x	1
01	0	1	X	1
11	1	1	x	x
10	1	1	x	x

fa = (A + C + B + ~D)(A + ~B + C + D)

VII. Comparator

- In the digital combination lock circuit, the comparator plays a crucial role in checking whether the sequence of digits entered (stored in the display memory) matches the preset password (stored in the password memory).

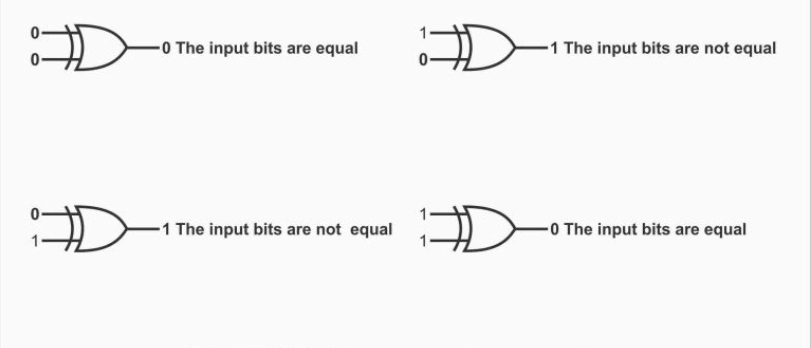
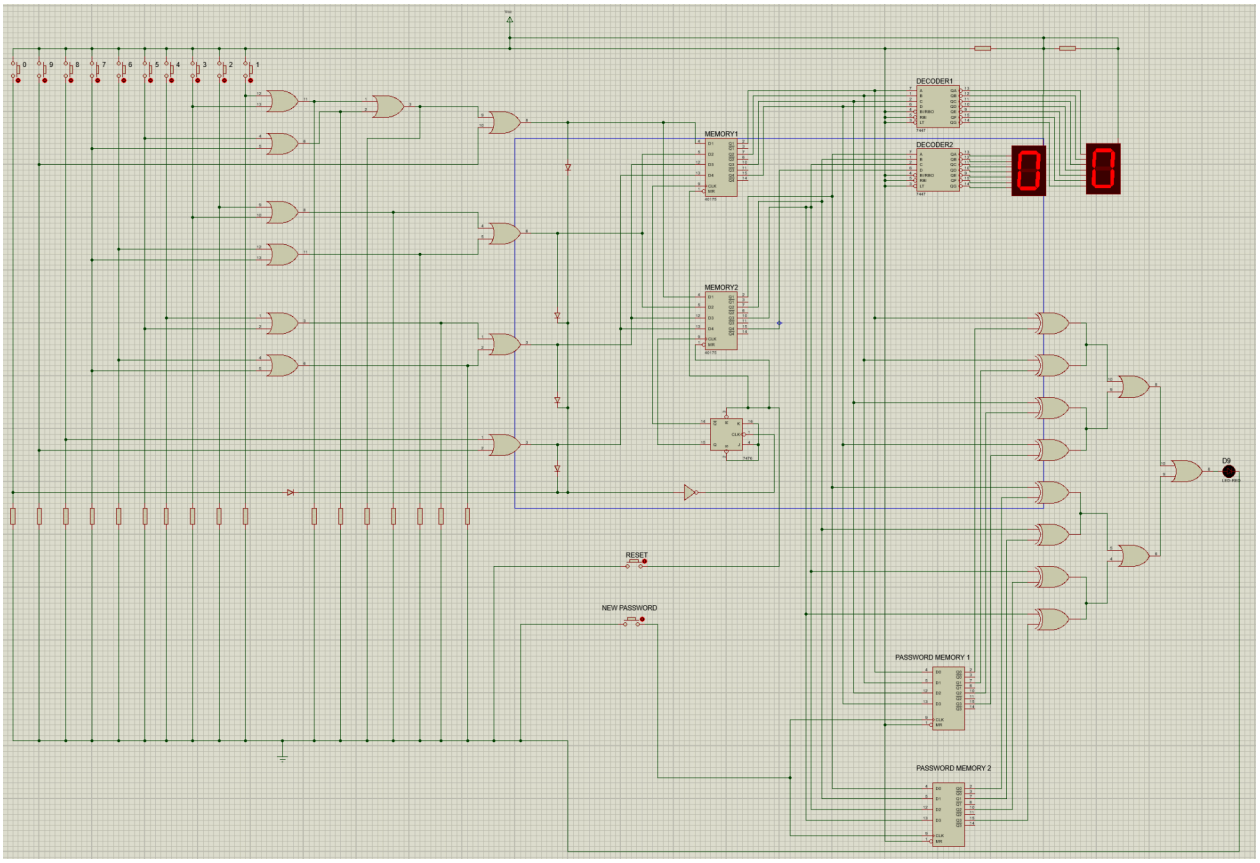


Fig 3.22 Basic comparator operation

- Each corresponding bit from the display memory and the password memory is connected to a **NOR gate**. This gate outputs a logic **0** when both inputs are equal, meaning the bits match.
- If **all bit pairs** match, the output of the **OR** gate will be **low (logic 0)**, indicating that the entered password is correct.

VIII. Simulation and applications

1. Simulation



- The simulation takes place in Proteus.
- When entering the wrong password, the LED indicating the lock is still on.
- When the right password is entered, the LED turns off, implying the lock has been turned off.

2. **Applications**
    - Digital combination lock is usually used in home security, office access systems, locker systems, etc...

IX. Conclusion

- We have applied knowledge of combinational and sequential logic circuits, especially flip-flops, logic gates, and decimal-to-BCD converters, to build a digital combination lock.
- Although there are still some limitations, such as the lack of an alarm function after multiple incorrect attempts or the ability to store multiple passwords, the project has achieved its objectives and demonstrated the practicality of logic circuits in security and control systems.

X. Time schedule

- 25th April, 2025: Hồng Sơn offered the Security Alarm System, but it uses the Sensor components, which is very complicated to write the function and apply it to the system.
- 26th April, 2025: Duy Bảo offered the Electronic Door Lock System, using only flip-flops and logic gates to build the system.
- 27th April, 2025: The group accepted the idea and Khoa offered the register, but it was hard for the whole team to understand and build the circuit.
- 28th April, 2025: Duy Bảo demonstrated the Digital Lock System with an Arduino board, but the code didn't run.
- 29th April, 2025: Hồng Sơn displayed a basic circuit with various blocks, we understood and chose it to be our project.
- 30th April, 2025: We divided tasks, saved each content to Word files.
- 1st May, 2025: we finished the report on Google Docs and the slide for presentation on Google Slides.
- 2nd May, 2025: we summarized the contents of the report, presented the Slides to have a successful presentation.

***References:**

- Nino123 and Instructables (2017) *Digital Combination Lock!*, *Instructables*. Available at: https://www.instructables.com/Digital-Combination-Lock (Accessed: 27 April 2025).
- Stephen Brown and Zvonko Vranesic (2013), Fundamentals of Digital Logic with Verilog Design (3rd ed), McGraw Hill.
- https://www.alldatasheet.com/html-pdf/12635/ONSEMI/74LS175/367/2/74LS175.html (datasheet IC 74LS175)
- https://www.auhd.edu.ye/upfiles/elibrary/Azal2020-01-22-12-16-48-75016.pdf (Digital Design – M. Morris Mano & Michael D. Ciletti (5th Edition)
- https://en.wikipedia.org/wiki/Flip-flop_(electronics)#JK_flip-flop (JK Flip-Flop, D Flip-Flop)